

ExerciseVideo.js — Fundamentos

Explicación detallada del componente de reproducción de video (GymBro App)

Es un componente chico (18 líneas) pero usa una API nueva de Expo (expo-video, parte del SDK 57) que vale la pena entender bien porque es distinta a como se manejaba video en versiones viejas de Expo (expo-av, ya deprecado).

1. Los imports (líneas 1-3)

```
import { StyleSheet } from 'react-native';
import { VideoView, useVideoPlayer } from 'expo-video';
```

expo-video es el módulo oficial de Expo para reproducir video (reemplazó a Video de expo-av). Expone dos cosas que se usan siempre juntas:

- **useVideoPlayer:** un hook que crea y controla un “reproductor” (el motor que descarga/decodifica el video).
- **VideoView:** el componente visual (la superficie donde se dibuja el video en pantalla).

Es el mismo patrón que “player” separado de “vista” que usan otras librerías de media: la lógica de reproducción (play/pause/loop/volumen) vive en un objeto player, y la vista es “tonta”, solo dibuja lo que ese player está reproduciendo.

2. El hook useVideoPlayer (líneas 6-9)

```
const player = useVideoPlayer(video, (player) => {
  player.loop = false;
  player.play();
});
```

- **Primer argumento (video):** la fuente del video. En este proyecto, mirando exercises.js, es un string con una URL (ej: 'https://tu-video.com/press-banca.mp4' — son placeholders, todavía no hay videos reales cargados). useVideoPlayer también acepta un require('./local.mp4') para videos empaquetados en la app, pero acá el dato viene de la “base de datos” simulada como URL remota.
- **Segundo argumento:** una función de configuración inicial (callback de setup) que Expo ejecuta una sola vez, apenas se crea el player, y te pasa el player recién creado para configurarlo: player.loop = false (que no se repita solo al terminar) y player.play() (arranca la reproducción automáticamente apenas está listo, autoplay).

Punto importante para tu defensa: useVideoPlayer es un hook, así que sigue las reglas de hooks de React — se re-ejecuta si video cambia entre renders (crea un player nuevo), y React se encarga de destruir el player viejo cuando el componente se desmonta (por ejemplo, al cerrar el modal), liberando los recursos nativos de decodificación de video. Vos no tenés que hacer cleanup manual.

3. El componente visual VideoView (líneas 12-16)

```
<VideoView
  player={player}
  style={styles.video}
  nativeControls
/>
```

- **player={player}**: conecta esta vista con el player que creaste arriba. Es la misma relación que value/onChangeText en un input controlado, pero para video: el VideoView simplemente refleja el estado del player.
- **nativeControls**: le dice a Expo que muestre los controles nativos del sistema operativo (play/pause, barra de progreso, volumen, pantalla completa) en vez de que vos tengas que construir tus propios botones. Es una prop booleana abreviada (nativeControls equivale a nativeControls={true}).
- Al no pasar más props de control, este componente es “recibí un video y reproduzco con controles estándar” — no hay lógica de pausa manual, tracking de progreso, ni nada custom.

4. Los estilos (líneas 20-27)

```
video: {
  width: '100%',
  aspectRatio: 16 / 9,
  backgroundColor: '#000000',
  borderRadius: 7,
  alignSelf: 'center',
},
```

- aspectRatio: 16/9 es la relación de aspecto estándar de video horizontal. En vez de fijar un height en píxeles (que rompería en distintos tamaños de pantalla), le decís a RN “el alto se calcula automáticamente para mantener esta proporción según el ancho disponible” — por eso funciona bien tanto en isLandscape como en portrait sin necesitar lógica de useResponsiveLayout acá adentro.
- backgroundColor: '#000000' cubre el espacio mientras el video carga o si tiene barras negras (letterboxing).

5. Cómo se integra con el resto de la app

En ver_dia_x_user.js (líneas 245-249):

```
{selectedExercise && modalType === 'video' && (
  <ExerciseVideo video={selectedExercise.video} />
)}
```

- Está dentro de un Modal (línea 227) que se abre cuando el usuario toca un ejercicio del día y elige ver el video.
- Renderizado condicional doble: solo se monta ExerciseVideo si hay un selectedExercise **y** el modal está en modo 'video' (hay otro modo 'instructions' que muestra texto en vez de video, mismo modal, contenido distinto).
- **Detalle clave de performance/UX**: como ExerciseVideo (y por lo tanto useVideoPlayer) solo se monta cuando el modal está abierto y en modo video, el video no se descarga ni reproduce en segundo plano — se crea el player recién al abrir el modal, y se destruye (gracias al cleanup automático del hook) al cerrarlo o cambiar de ejercicio. Esto evita gastar datos/batería reproduciendo videos que el usuario no está viendo.
- selectedExercise.video es el string URL que viene de exercises.js — cada ejercicio de la rutina tiene su propio campo video.

Resumen para el oral

“ExerciseVideo es un wrapper delgado sobre expo-video: separa la lógica de reproducción (useVideoPlayer, que crea el player y lo configura para autoplay sin loop) de la vista (VideoView, que solo dibuja ese player con controles nativos). Es un componente 'tonto' — no maneja su propio estado de reproducción, delega todo en el player de Expo. Se monta bajo demanda dentro de un modal, así que el video solo existe en memoria mientras el usuario lo está mirando.”